Integration Testing Report

StreetRace Manager System

---

1. System Overview

The StreetRace Manager is a command-line system built to manage underground street races, crew members, vehicles, and missions. It is composed of six mandatory modules and two additional custom modules, each responsible for a distinct area of the system. The modules communicate with one another through well-defined function calls, and all cross-module interactions follow strict business rules.

The table below summarises every module in the system, its file name, and its primary responsibility.

Module File	Type	Description
registration.py	Mandatory	The entry point of the system. Registers new crew members with unique IDs so their actions can be tracked across all other modules.
crew_management.py	Mandatory	Acts as the HR department. Assigns roles such as driver or mechanic to registered members and maintains their skill level records.
inventory.py	Mandatory	Serves as both the bank and the garage. Tracks the team's cash balance, spare parts stock, and the physical condition of every car.
race_management.py	Mandatory	The event coordinator. Creates races and enforces strict eligibility checks, ensuring only registered drivers in undamaged cars are allowed to compete.
results.py	Mandatory	The referee module. Records race finishing times, determines podium positions, and distributes prize money to the correct inventory accounts.
mission_planning.py	Mandatory	Manages side operations such as deliveries or rescues. Validates that the required crew roles and headcount are available before a mission can begin.
garage.py	Custom	The repair shop. Allows mechanics to restore damaged cars using team cash and spare parts, and to apply performance upgrades after races.
leaderboard.py	Custom	The hall of fame. Pulls historical race data from the Results module and calculates career wins, podium streaks, and season-long driver rankings.

2. Custom Additional Modules

Two extra modules were designed and implemented to extend the system's functionality and to create a richer set of inter-module interactions for integration testing.

2.1 Garage Module (garage.py)

The Garage module acts as the team's repair shop, allowing mechanics to restore damaged cars and apply performance upgrades after races.

Responsibilities:

- Scheduling and completing car repairs after race damage.
- Applying top-speed performance upgrades using spare parts from Inventory.
- Enforcing that only crew members with the mechanic role may perform repairs.

Module interactions:

- Communicates with Inventory to deduct cash for repair costs and to consume spare parts for upgrades.
- Communicates with Crew Management to verify that the assigned crew member holds the mechanic role before scheduling any work.

Rationale: Without a repair mechanism, a car damaged in a race would be permanently removed from competition. The Garage module brings cars back into the game while ensuring the team has sufficient resources and the correct personnel to carry out the work.

2.2 Leaderboard Module (leaderboard.py)

The Leaderboard module tracks every driver's performance over time, building a season-long competitive ranking from historical race data.

Responsibilities:

- Calculating total career wins, top-3 podium finishes, total races entered, and active winning streaks for each driver.
- Providing a dynamically sorted standings view at any point during the season.

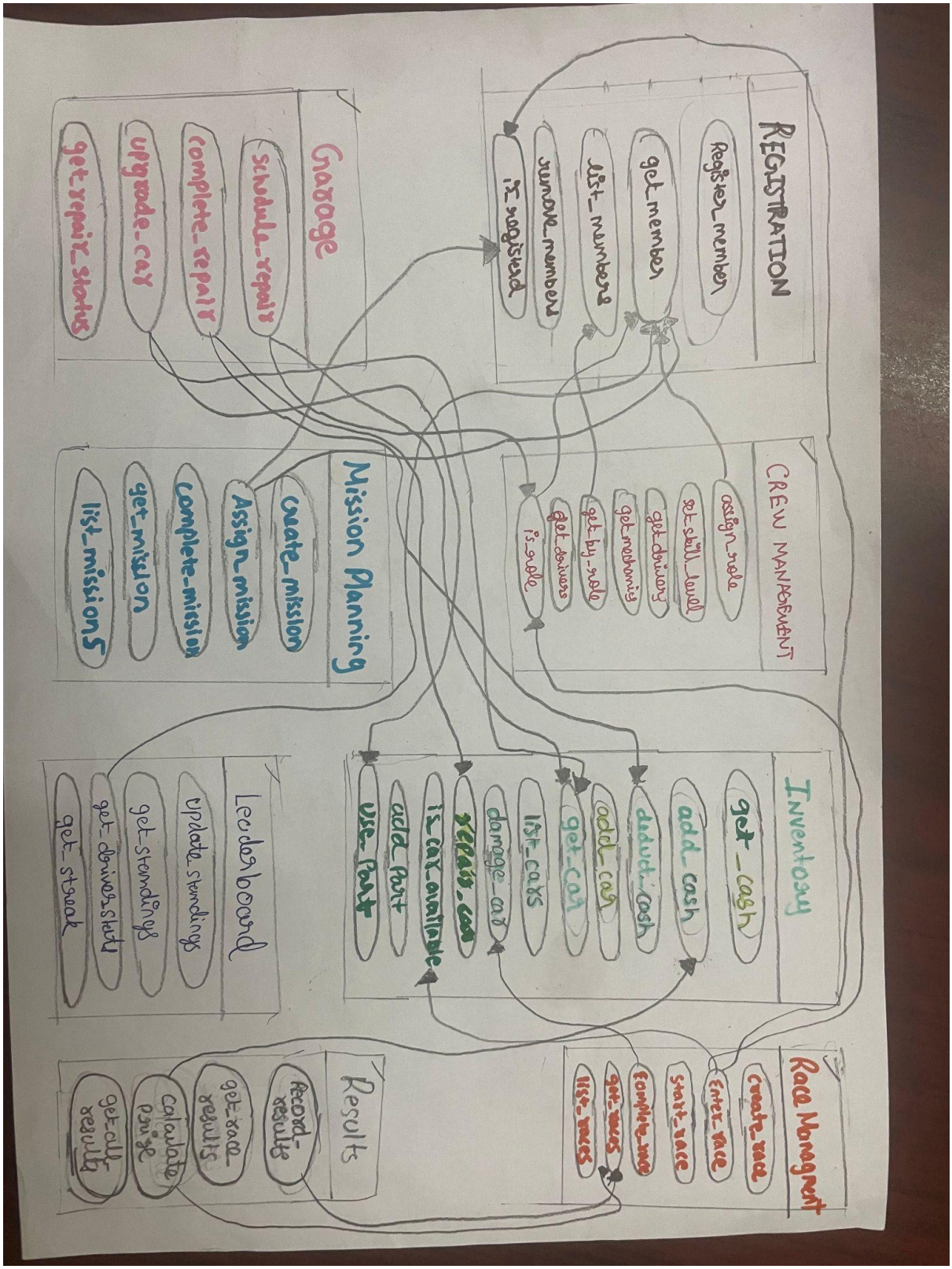
Module interactions:

- Pulls race result data directly from the Results module every time a race concludes.
- Queries the Registration module to resolve driver IDs into display names for the standings output.

Rationale: The Leaderboard gives the simulation a long-term competitive dimension. It also enables testing of a difficult edge case: what happens to historical statistics if an active driver is removed from the registry mid-season, as explored in Bug 5.

3. Call Graph

The call graph below was drawn by hand and shows every inter-module function call within the StreetRace Manager system. Each node represents a single function and each arrow indicates a call from one module to another. The diagram was photographed and is attached below as part of this report.



For reference, the complete list of inter-module calls is also presented in tabular form below.

Caller Module	Caller Function	→	Callee Module	Callee Function
Crew Management	_validate_member	→	Registration	is_registered
Crew Management	assign_role	→	Registration	get_member
Crew Management	get_by_role	→	Registration	list_members
Crew Management	is_role	→	Registration	get_member
Race Management	enter_race	→	Registration	is_registered
Race Management	enter_race	→	Crew Management	is_role
Race Management	enter_race	→	Inventory	is_car_available
Race Management	complete_race	→	Inventory	damage_car
Results	record_result	→	Race Management	get_race
Results	calculate_prize	→	Race Management	get_race
Results	calculate_prize	→	Inventory	add_cash
Mission Planning	assign_mission	→	Registration	is_registered
Mission Planning	assign_mission	→	Registration	get_member
Garage	schedule_repair	→	Inventory	get_car
Garage	schedule_repair	→	Crew Management	is_role
Garage	complete_repair	→	Inventory	deduct_cash
Garage	complete_repair	→	Inventory	repair_car
Garage	upgrade_car	→	Inventory	get_car
Garage	upgrade_car	→	Inventory	use_part
Leaderboard	get_driver_stats	→	Registration	get_member

4. Integration Test Design

The test suite consists of 39 test cases across eight groups. Each case validates interactions between two or more modules, verifying data flow and business rule enforcement at the integration boundary rather than testing individual module logic in isolation.

4.1 Crew and Registration Verification (Tests 1–5)

These tests confirm that Crew Management correctly enforces the registration prerequisite before performing any role or skill assignment operations.

#	Test Method	Modules	Scenario & Purpose	Result
1	test_assign_role_requires_registration	CrewManagement, Registration	Ensures that attempting to assign a role to a member ID that does not exist in the registry	ValueError raised as expected; no issues found.

#	Test Method	Modules	Scenario & Purpose	Result
			raises a ValueError, enforcing the registration prerequisite.	
2	test_assign_role_to_registered_member	CrewManagement, Registration	Verifies that a fully registered member can have a role successfully assigned, confirming the happy-path flow between the two modules.	Role assigned correctly; no issues found.
3	test_set_skill_requires_registration	CrewManagement, Registration	Ensures that skill levels (1–10) cannot be set for a member ID that has not been registered, preventing orphaned skill records.	ValueError raised as expected; no issues found.
4	test_get_by_role_reads_registration_data	CrewManagement, Registration	Validates that filtering crew members by role correctly queries the Registration registry and returns only members matching the requested role.	Correct members returned; no issues found.
5	test_is_role_checks_registration	CrewManagement, Registration	Checks that the boolean helper function correctly validates whether a given member holds a specific role such as driver or mechanic.	Boolean result accurate; no issues found.

4.2 Race Entry Validation (Tests 6–10)

These tests verify the eligibility gates applied by Race Management before a driver or car is admitted to a race.

#	Test Method	Modules	Scenario & Purpose	Result
6	test_register_driver_then_enter_race	Registration, RaceManagement	Tests the standard successful flow of registering a driver and then entering them into a race, confirming both modules cooperate correctly.	Race entry successful; no issues found.
7	test_enter_race_without_registration	Registration, RaceManagement	Ensures that an unregistered member ID cannot be entered into a race, verifying the registration gate in Race Management.	ValueError raised as expected; no issues found.

#	Test Method	Modules	Scenario & Purpose	Result
8	test_enter_race_with_non_driver_role	CrewManagement, RaceManagement	Prevents crew members with a non-driver role, such as a mechanic or strategist, from being entered as competitors in a race.	ValueError raised as expected; no issues found.
9	test_enter_race_with_damaged_car	Inventory, RaceManagement	Ensures the eligibility check in Race Management correctly blocks a car that is flagged as damaged from being entered into a race.	ValueError raised as expected; no issues found.
10	test_enter_race_with_nonexistent_car	Inventory, RaceManagement	Ensures that a fabricated or non-existent car ID cannot be used to enter a race, validating the inventory lookup in Race Management.	ValueError raised as expected; no issues found.

4.3 Race Results and Inventory Economics (Tests 11–15)

These tests validate the financial and state consequences of running and completing a race, spanning Race Management, Results, and Inventory.

#	Test Method	Modules	Scenario & Purpose	Result
11	test_race_completion_can_damage_cars	RaceManagement, Inventory	Verifies that the random chance of a car's condition dropping after a race completion is correctly applied and reflected in Inventory.	Damage flag set correctly; no issues found.
12	test_damaged_car_from_race_blocks_next_race	RaceManagement, Inventory	Ensures a car that was damaged in Race A is correctly locked out of Race B, confirming the cross-race eligibility check.	Car blocked as expected; no issues found.
13	test_record_result_validates_race_status	RaceManagement, Results	Prevents recording finishing times for races that have not yet been formally started, enforcing correct state transitions.	ValueError raised as expected; no issues found.
14	test_race_results_update_inventory_cash	Results, Inventory	Verifies that 1st, 2nd, and 3rd place prize payouts are correctly deposited into the respective drivers'	Cash balances updated correctly; no issues found.

#	Test Method	Modules	Scenario & Purpose	Result
			Inventory cash balances after a race.	
15	test_record_result_validates_driver_was_in_race	Results, RaceManagement	Ensures that a driver who never entered a race cannot be recorded as having finished it, preventing phantom result entries.	ValueError raised as expected; no issues found.

4.4 Mission Planning Checks (Tests 16–19)

These tests ensure that Mission Planning correctly validates crew availability and role requirements before any mission can be dispatched.

#	Test Method	Modules	Scenario & Purpose	Result
16	test_assign_mission_with_correct_roles	MissionPlanning, Registration	Happy path: verifies that a mission can be successfully assigned when all required roles are covered by the listed crew members.	Mission assigned successfully; no issues found.
17	test_assign_mission_missing_required_role	MissionPlanning, Registration	Verifies that mission deployment halts with a ValueError when a required role is not available among the assigned crew.	ValueError raised as expected; no issues found.
18	test_assign_mission_all_roles_covered	MissionPlanning, Registration	Asserts that a mission completes successfully when every required role is accounted for across the assigned crew members.	Mission assigned successfully; no issues found.
19	test_assign_mission_with_unregistered_member	MissionPlanning, Registration	Prevents unregistered member IDs from being sent on missions, verifying the registration validation in Mission Planning.	ValueError raised as expected; no issues found.

4.5 Garage Maintenance and Upgrades (Tests 20–26)

These tests cover the full range of Garage operations, from scheduling repairs to applying upgrades, verifying all interactions with Inventory and Crew Management.

#	Test Method	Modules	Scenario & Purpose	Result
20	test_schedule_repair_requires_mechanic_role	Garage, Registration	Ensures that only crew members with the	ValueError raised for

#	Test Method	Modules	Scenario & Purpose	Result
			mechanic role can be assigned to a repair job, blocking non-mechanics from scheduling repairs.	non-mechanic; no issues found.
21	test_schedule_repair_checks_car_in_inventory	Garage, Inventory	Verifies that a repair cannot be scheduled for a car ID that does not exist in the Inventory, preventing repairs on phantom vehicles.	ValueError raised as expected; no issues found.
22	test_complete_repair_deducts_cash_and_fixes_car	Garage, Inventory	Validates that completing a repair correctly deducts the repair cost from the cash balance and restores the car's condition to 100.	Cash deducted and condition restored; no issues found.
23	test_repair_then_car_can_race_again	Garage, Inventory, RaceManagement	Full pipeline test: damages a car in a race, repairs it through the Garage, then confirms it is eligible to enter a subsequent race.	Car re-entered successfully after repair; no issues found.
24	test_upgrade_car_uses_part_from_inventory	Garage, Inventory	Verifies that applying an upgrade correctly increases the car's top speed and decrements the spare parts count in Inventory.	Top speed increased and parts decremented; no issues found.
25	test_upgrade_fails_without_part	Garage, Inventory	Ensures that a performance upgrade is rejected when the required spare part is not present in the Inventory.	ValueError raised as expected; no issues found.
26	test_upgrade_fails_on_damaged_car	Garage, Inventory	Prevents performance upgrades from being applied to a car that still has unrepaired structural damage, enforcing repair-first logic.	ValueError raised as expected; no issues found.

4.6 Leaderboard Statistics (Tests 27–28)

These tests confirm that driver statistics in the Leaderboard are correctly updated after race results are recorded.

#	Test Method	Modules	Scenario & Purpose	Result
27	test_race_results_update_leaderboard	Results, Leaderboard	Verifies that win and podium counters in the Leaderboard increment correctly after a race result is recorded through the Results module.	Counters incremented correctly; no issues found.
28	test_leaderboard_ranking_order	Results, Leaderboard	Ensures that get_standings() dynamically sorts drivers by total points and wins rather than insertion order, keeping the rankings accurate.	Standings returned in correct order; no issues found.

4.7 Multi-Module End-to-End Flows (Tests 29–31)

These large-scope tests sweep through multiple modules in sequence, simulating realistic complete workflows from start to finish.

#	Test Method	Modules	Scenario & Purpose	Result
29	test_full_race_flow	Reg, Crew, Inv, Race, Results, LB	End-to-end test sweeping through all six core modules sequentially: register a driver, assign a role, add a car, create and run a race, record results, and verify the leaderboard updates.	All modules interacted correctly; no issues found.
30	test_damage_repair_rerace_flow	Reg, Inv, Race, Garage	Traces the full lifecycle of a vehicle from purchase through a race that causes damage, through the Garage repair process, and back into an eligible racing state.	Lifecycle completed successfully; no issues found.
31	test_mission_after_race_damage_needs_mechanic	Race, Inv, Mission	A race damages a car; the subsequent mission planning step correctly flags that a mechanic is required before the mission can proceed.	Mechanic requirement enforced correctly; no issues found.

4.8 Bug Discovery and Edge-Case Tests (Tests 32–39)

The following tests were specifically designed to probe edge cases and boundary conditions that exposed five distinct bugs in the implementation. Each test is labelled with the bug it targets.

#	Test Method	Modules	Scenario & Purpose	Result
32	test_calculate_prize_twice_should_not_double_cash	Results, Inventory	Bug 1 — Ensures that calling calculate_prize() on the same race a second time does not distribute the prize pool again, preventing cash duplication.	Bug confirmed; fixed by introducing a _paid_races set tracker.
33	test_prize_only_distributed_once_total_check	Results, Inventory	Bug 1 — Asserts financial exactness across the wider economy state to confirm that repeated prize calls do not alter the total cash balance.	Bug confirmed; fix verified by checking exact final balance.
34	test_damage_already_damaged_car_should_not_reduce_condition	Inventory	Bug 2 — Prevents a car that is already marked as damaged from accumulating further condition penalties, which would push its condition below zero.	Bug confirmed; fixed by adding an early-exit guard in damage_car().
35	test_damage_car_three_times_condition_should_not_hit_zero	Inventory	Bug 2 — Further bounds-checking on the damage function to ensure repeated calls do not compound into an unrepairable negative condition value.	Bug confirmed; fix verified across multiple consecutive calls.
36	test_driver_should_not_enter_two_open_races	RaceManagement	Bug 3 — Ensures that a driver ID cannot be entered into two different open races simultaneously, closing the driver ghosting bypass.	Bug confirmed; fixed by expanding the validation scope in enter_race().
37	test_car_should_not_enter_two_open_races	RaceManagement, Inventory	Bug 3 — Verifies that the same car asset cannot appear in two races at once, preventing vehicle ghosting across concurrent events.	Bug confirmed; fix verified alongside the driver ghosting fix.
38	test_mission_requires_multiple_same_role_allows_single	MissionPlanning, Reg	Bug 4 — Checks for set-collapsing logic that allowed a mission requiring two mechanics to proceed with only one, due to Python set() deduplication.	Bug confirmed; fixed by replacing set() arithmetic with list .remove() iteration.

#	Test Method	Modules	Scenario & Purpose	Result
39	test_remove_driver_in_active_race_fails_results	Registration, Leaderboard	Bug 5 — Explores system stability when an active driver is removed mid-season, ensuring the Leaderboard does not crash with a KeyError on stale data.	Bug confirmed; fixed by implementing a soft-delete (active flag) in Registration.

5. Summary of Bug Fixes

The integration test suite identified five bugs, all of which have been corrected and committed to the repository in the format "Error #: <What Was Changed>". The table below summarises each bug, its root cause, and the fix that was applied.

ID	Name	Module	Problem	Fix Applied
Bug 1	Infinite Prize Payout	results.py	calculate_prize() could be called multiple times on the same race, distributing the prize pool repeatedly and inflating all balances indefinitely.	Added a _paid_races = set() state tracker. The method now checks if race_id in self._paid_races and returns immediately if so, ensuring the pool is paid out exactly once.
Bug 2	Double Damage Penalty	inventory.py	If a car was already flagged as damaged, calling damage_car() again would subtract a further 30 condition points, eventually dropping the value below zero.	Added an early-exit guard clause — if car['is_damaged']: return — at the top of the function to prevent any duplicate condition penalties.
Bug 3	Duplicate Race Entries	race_management.py	A single driver or car could be registered into two entirely separate open races simultaneously, allowing impossible concurrent participation.	Expanded the validation scope in enter_race() to loop through all open and in_progress races system-wide before accepting a new entry, ensuring no driver or car appears twice.
Bug 4	Mission Headcount Bypass	mission_planning.py	A mission requiring multiple members of the exact same role (e.g. two mechanics) could start with only one, because Python's set() collapsed duplicate role requirements into a single entry.	Replaced the flawed set() arithmetic with a strict list iteration. The code now makes a .copy() of the required roles list and .remove()s them one by one, enforcing exact quantities.
Bug 5	Stale Data Leaderboard Crash	registration.py	Calling remove_member() physically deleted a driver's dictionary record. If the Leaderboard later attempted to fetch their name from historical race	Implemented a soft-delete approach by adding an 'active': True/False flag to each member record. Removing a member now only sets the flag to False, preserving historical data for the

ID	Name	Module	Problem	Fix Applied
			statistics, the system crashed with a KeyError.	Leaderboard while blocking new registrations or role assignments.